INTRODUCTION
TO
BASIC PROGRAMMING

Outline

1. Basic Concepts of Programming
2. Programming Language
3. C Language Introduction

Basic Concepts

Algorithm vs. Coding vs. Programming

ALGORITHM	*CODING*	*PROGRAMMING*
a precise step-by-step set of instructions to solve a problem	writing instructions in a language that can be understood by a machine or computer	The process of creating algorithms and turning them into code

Basic Concepts

Programming Language



Instructions

Program

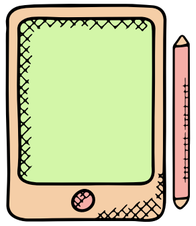


Programmer

Basic Concepts

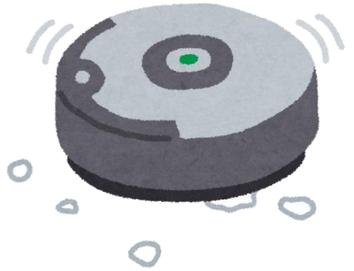
- **A program is a set of instructions that a computer follows to perform a task.**
- **Computers and electronic devices need programs to tell them what to do.**





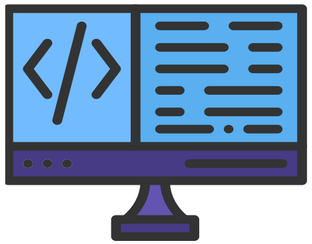
Computer programs are everywhere

Programming is not just about conventional computer systems anymore. The world has become increasingly digital, and almost everything runs on software. Programs are now incorporated into devices such as mobile phones and tablets, labour-saving equipment around the home, and even in transportation systems.





Programming Language



A programming language is a set of words and symbols that allows people to write instructions for a computer.

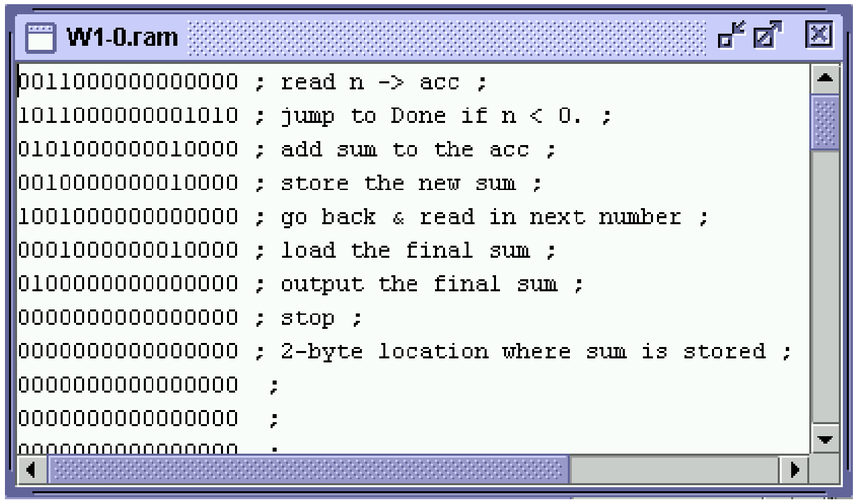
Types of Programming Languages

- **Low-Level Languages: A language that is close to the machine, difficult for humans to understand, but very fast and efficient.**
- **High-Level Languages: Closer to human language, easy to read, and portable (runs on many systems).**

Programming Language

Low-Level Languages

Machine Code: The native language of computers is binary, represented by 0s and 1s.



Assembly Language: Using short codes (mnemonics) to replace binary.

```
; ------------------------------------------------------------------------------
; Writes "Hello, World" to the console using only system calls. Runs on 64-bit Linux only.
; To assemble and run:
;
;     nasm -felf64 hello.asm && ld hello.o && ./a.out
; ------------------------------------------------------------------------------

          global    _start

          section   .text
_start:   mov       rax, 1                  ; system call for write
          mov       rdi, 1                  ; file handle 1 is stdout
          mov       rsi, message            ; address of string to output
          mov       rdx, 13                 ; number of bytes
          syscall                           ; invoke operating system to do the write
          mov       rax, 60                 ; system call for exit
          xor       rdi, rdi                ; exit code 0
          syscall                           ; invoke operating system to exit

          section   .data
message:  db        "Hello, World", 10      ; note the newline at the end
```

https://www.researchgate.net/figure/A-simple-machine-language-program_fig6_238779569
https://www.investopedia.com/terms/a/assembly-language.asp

Programming Language

High-Level Languages

High-level programming languages are designed to be easy to use without needing a detailed understanding of the computer hardware. They often use words that are similar to human language

HIGH-LEVEL LANGUAGES

- Relatively fast to write
- Relatively easy to understand
- Usually fast enough in execution
- Programs can be used on different hardware
- No need to understand hardware

Instruction to display some text

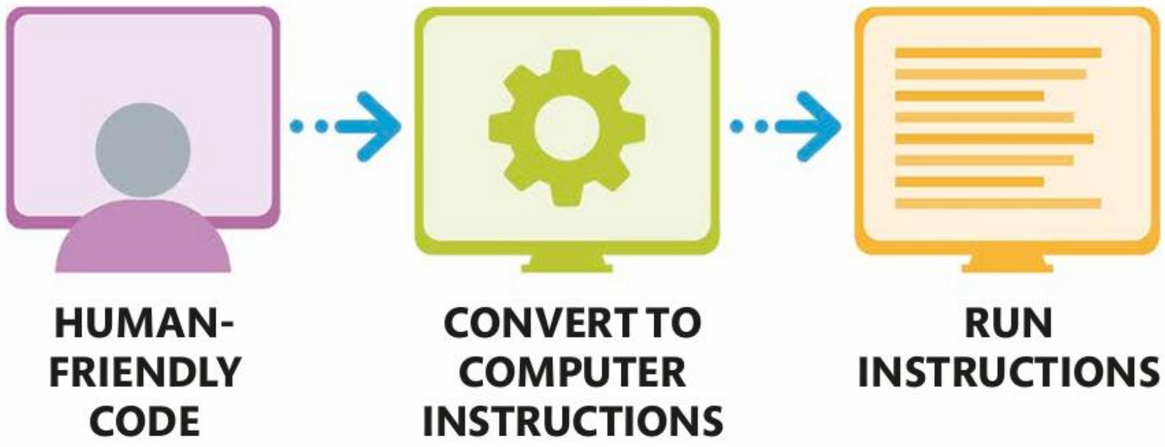
```
print("Hello!")
```

Python
A popular high-level language, Python is easy to read and write. This instruction puts the message "Hello!" on the screen.

Programming Language

How the computer understands a programming language?

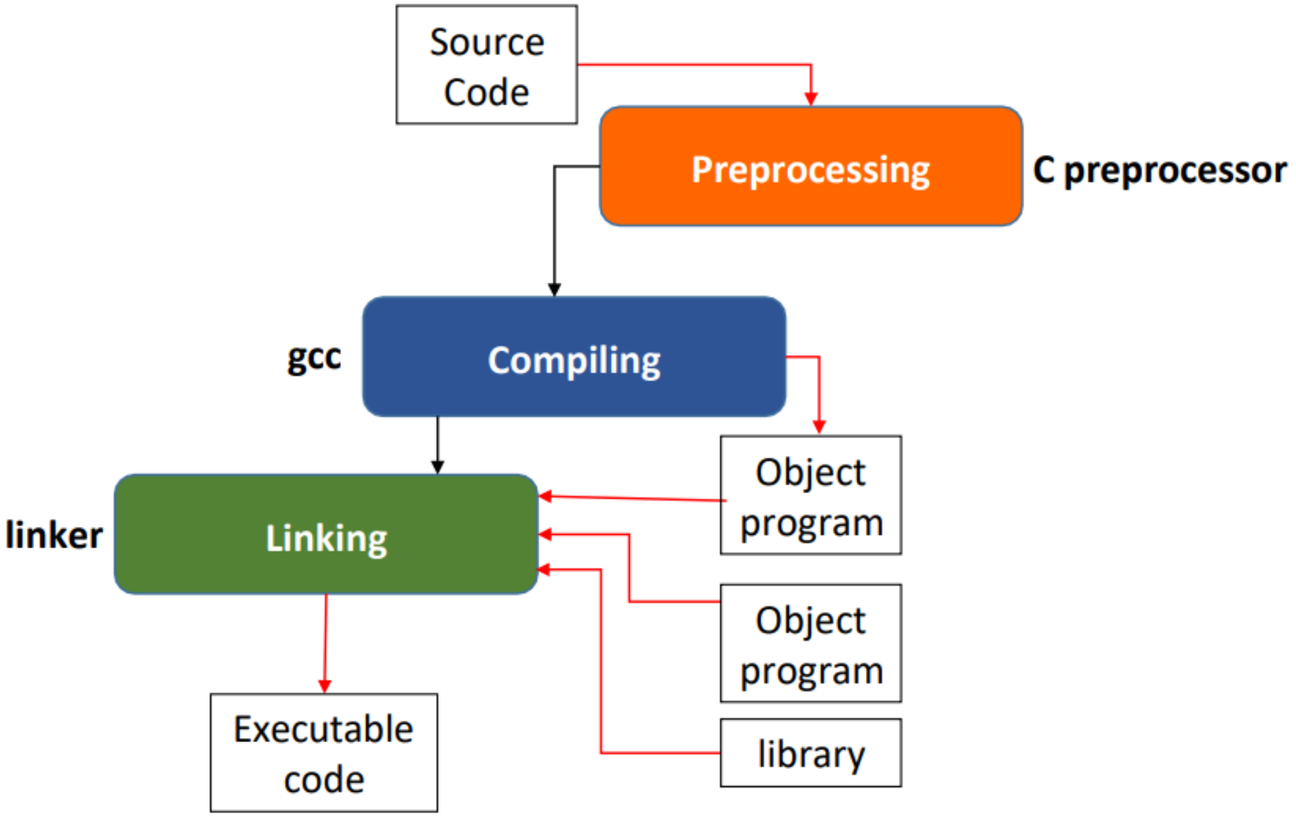
- Ultimately, all programs end up as machine code.
- Most programs are written in more human-friendly languages and need to be translated into raw bits so that they can be executed by a processor.
- An interpreter translates and executes the instructions as the program is running, while compilers translate the program in one go before it runs.



C Language Introduction

The C Development Cycle

- The C Development Cycle is the steps needed to run a C program on a computer.
- Because the computer only understands machine language (0s and 1s), the C code we write must be changed into an executable file first.



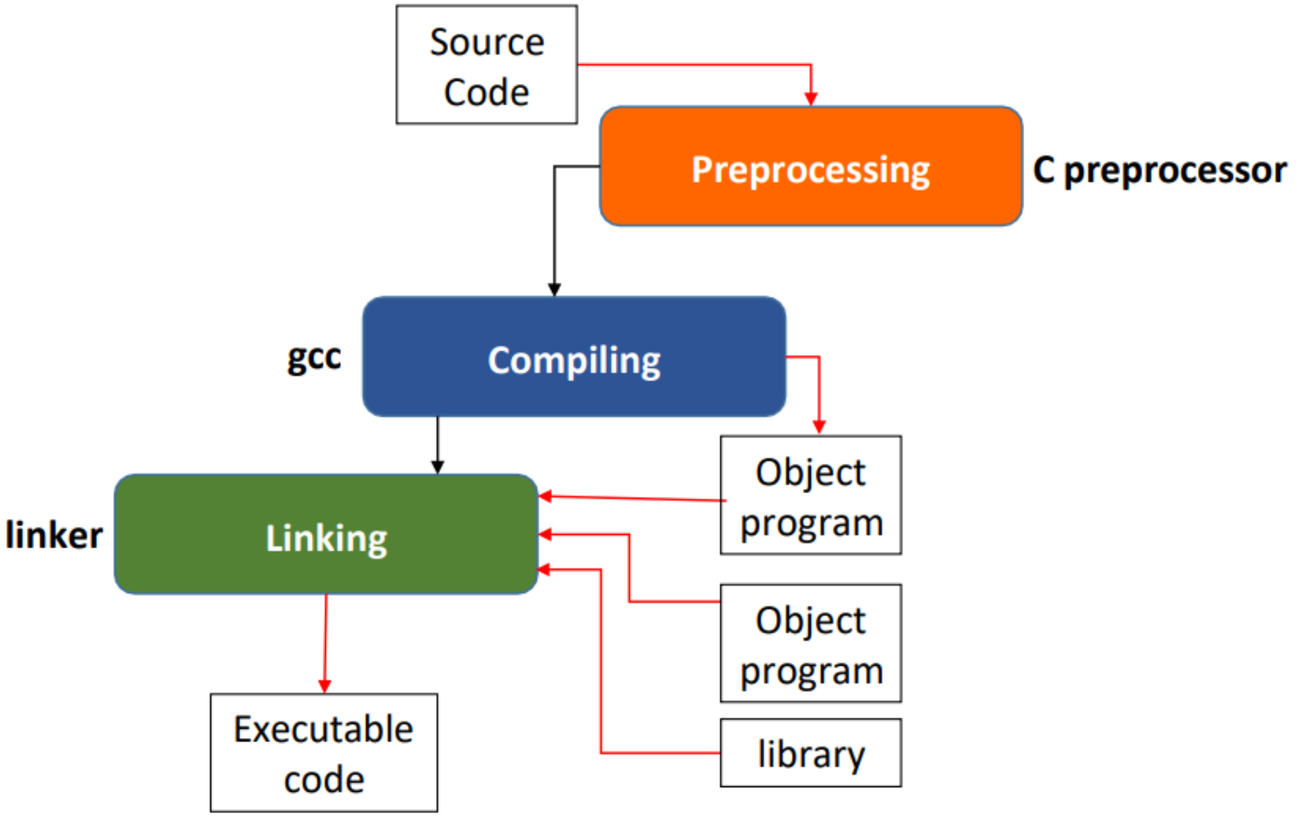
C Language Introduction

The C Development Cycle

Writing Source Code

Writing a C program in a file with the extension .c

Example: program.c

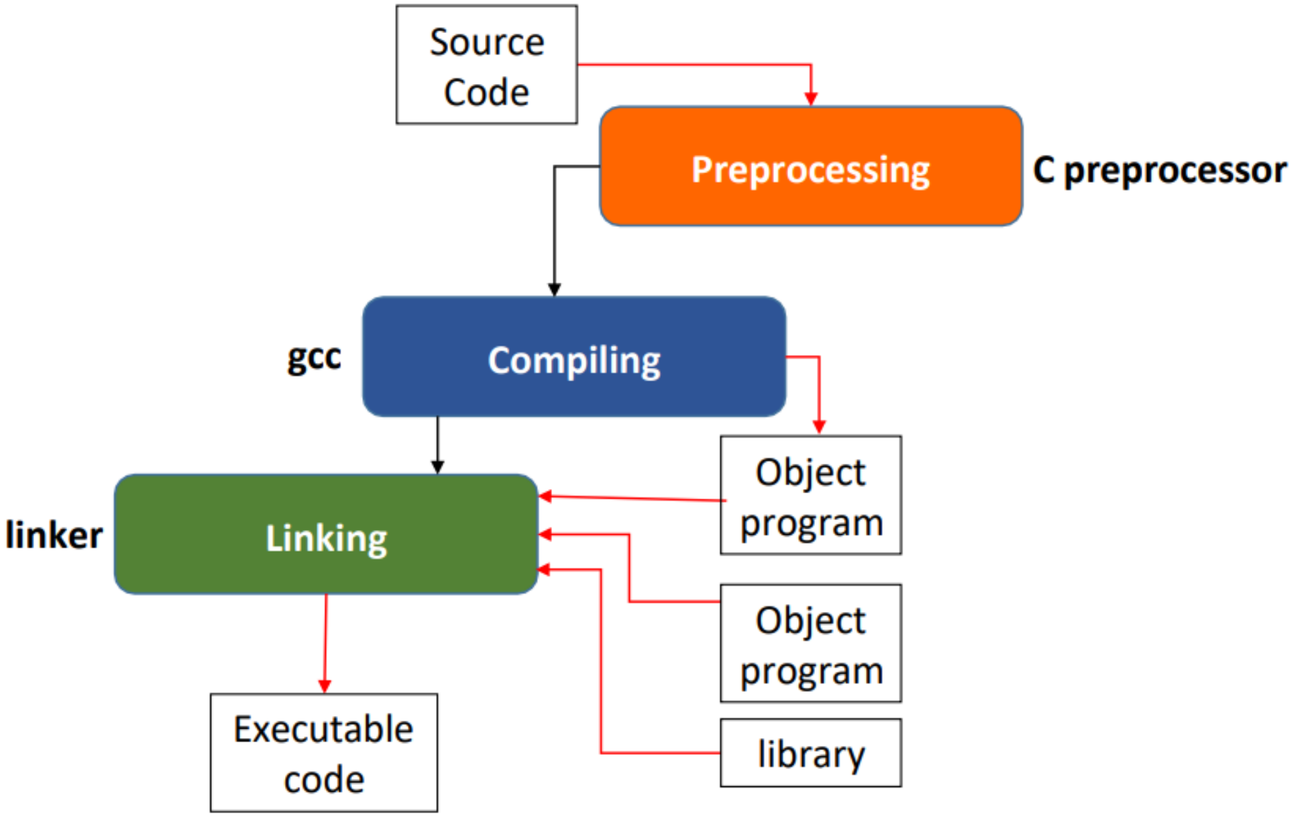


C Language Introduction

The C Development Cycle

Preprocessing (C Preprocessor)

- This step processes special commands such as #include or #define.
- The result is code that is ready to be compiled.

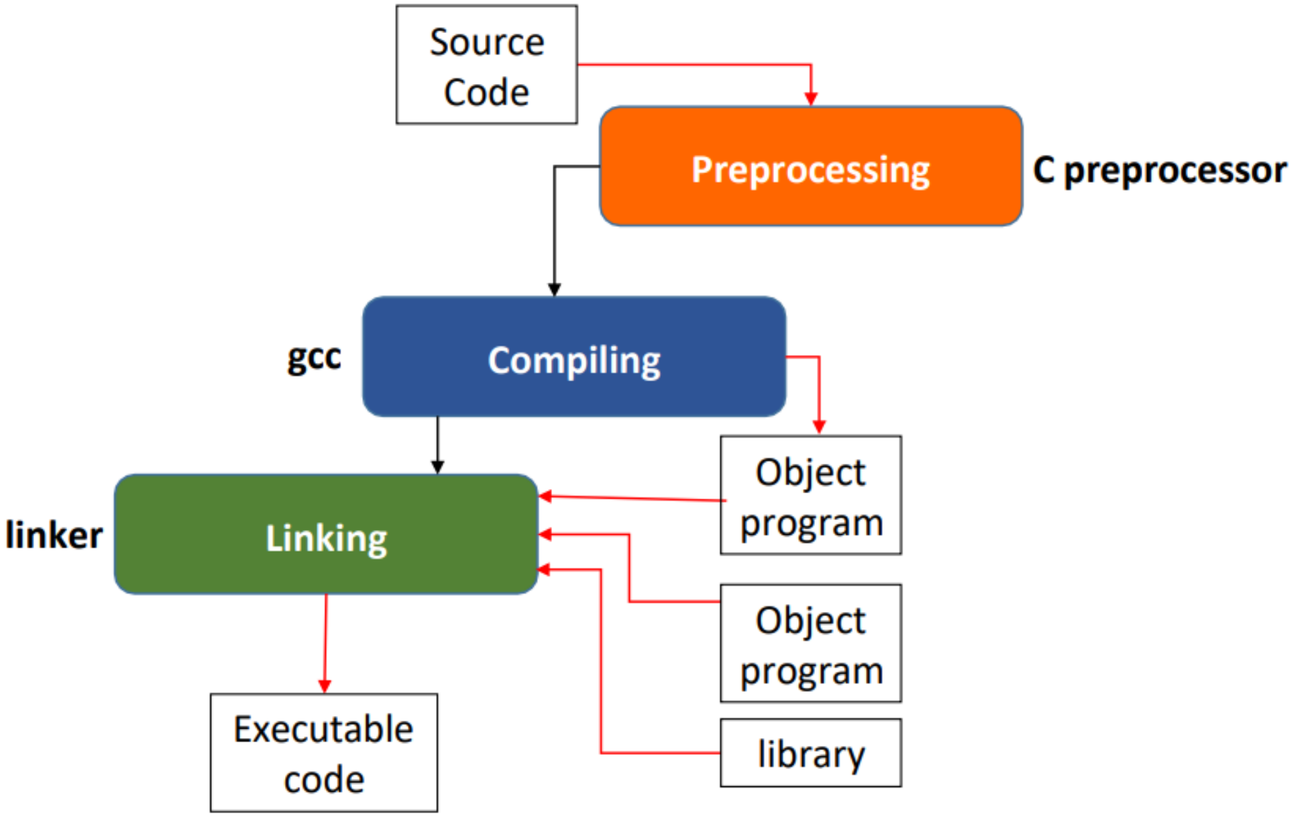


C Language Introduction

The C Development Cycle

Compiling

The compiler translates the C code into object program (a half-finished machine code that cannot run by itself).

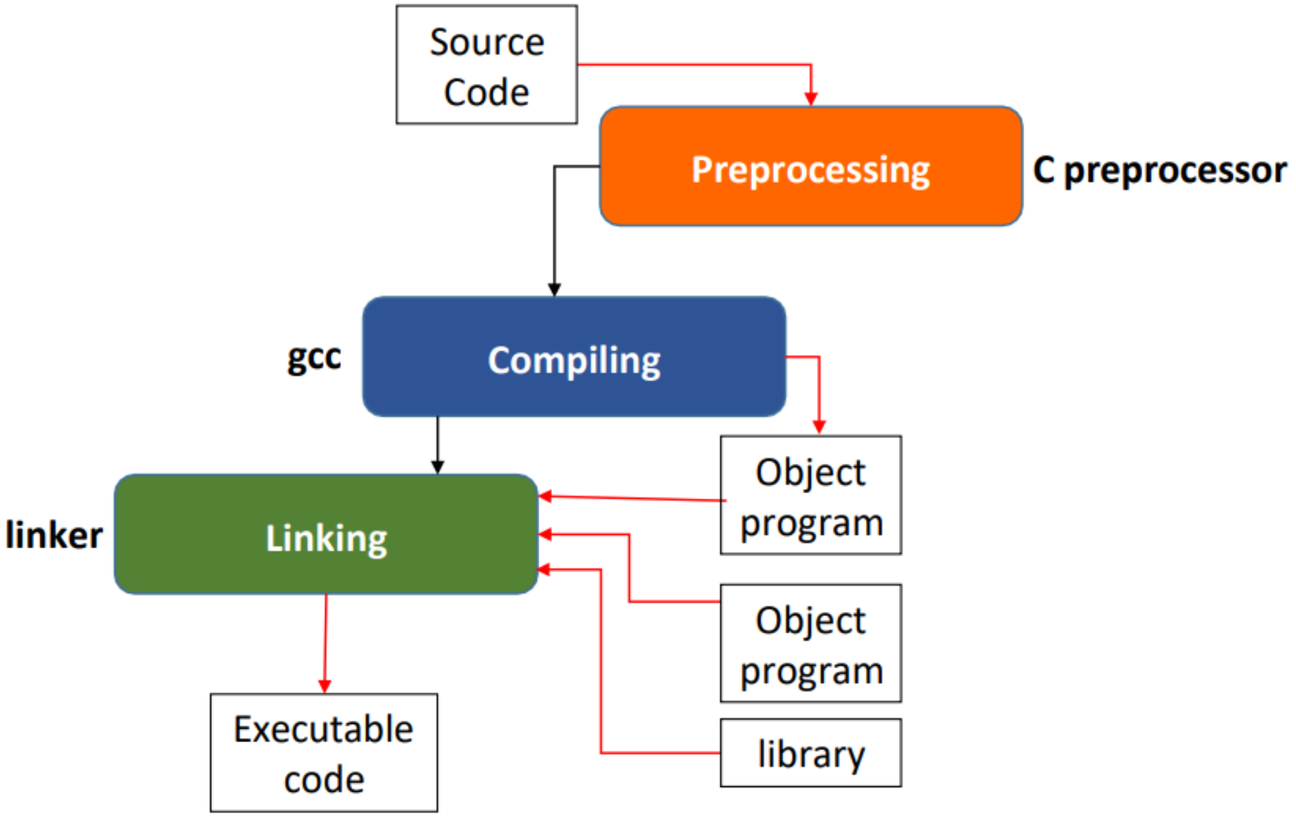


C Language Introduction

The C Development Cycle

Linking

- The linker combines the object code with the required libraries (such as stdio.h, math.h, etc.)
- The final result is an executable file (.exe on Windows)

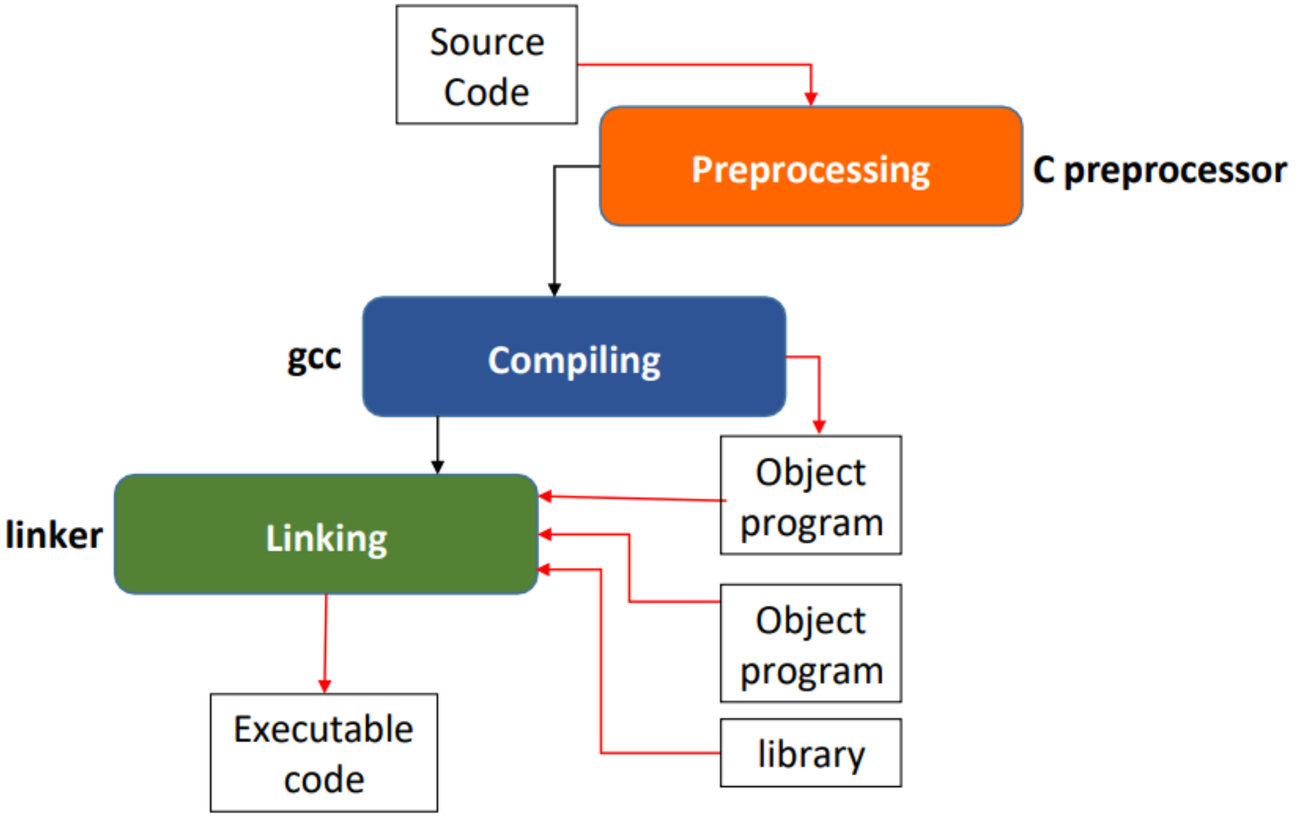


C Language Introduction

The C Development Cycle

Running the Executable

- The executable file is run by the operating system.
- The program shows the result according to the instructions written.



C Language Introduction

Algorithmic Notation

How to write an algorithm (steps to solve a problem) in a more structured and easy-to-understand form

Judul Berisi penjelasan singkat tentang tujuan algoritma/program.
Kamus Berisi data yang digunakan dalam program: variabel, konstanta, tipe datanya
Algoritma Berisi urutan instruksi/logika yang akan dijalankan

C Language Introduction

Structure of C Language

```c
/* Nama File    : hello.c
   Deskripsi    : Program menampilkan tulisan "Hello World" ke layar
*/

#include <stdio.h> // library input-output

int main()
{
    /* kamus */
    char pesan[] = "Hello World";

    /* algoritma */
    printf("%s\n", pesan);

    return 0;
}
```

C Language Introduction

Character Set in C

A collection of all characters that are recognized and can be used in the C programming language

```
Alphabet: Uppercase: A, B, ..., Z
          Lowercase: a, b, ..., z
Digits: 0, 1, 2, ..., 9
Special Characters:
+ - * / % < > = ! & | ( ) [ ] { } , ; : . # \ ~
White Spaces: blank space, tab, new line, etc...
```

C Language Introduction

Keywords

- Special words whose meanings are already defined by the C language.
- The C compiler recognizes these keywords and gives them special meaning.
- They are always written in lowercase.
- They cannot be used for any other purpose besides their original function.
- They cannot be used as identifiers (such as variable names, function names, array names, etc.).
- Using uppercase (for example INT, VOID) as variable names is not recommended because it can be confusing.

C Language Introduction

Keywords

Data types:

`int, float, double, char, void`

Control flow:

`if, else, switch, case, default, for, while, do, goto, continue, break, return`

Declarations/Storage:

`auto, extern, register, static, const, sizeof, typedef, struct, union, enum, volatile, signed, unsigned, short, long`

C Language Introduction

Identifiers

- Identifiers are names that are given to various program elements, such as variables, functions and arrays.
- Both uppercase and lowercase letters are permitted, though common usage favors the use of lowercase letters for most types of identifiers.
- Case sensitive, remember! (A != a)
- Keywords can not be used as identifiers

C Language Introduction

Identifiers

Valid	Invalid	Reason for Invalidity
`xyz`	`3rd`	Starts with a number
`volume`	`"Z"`	Illegal character (“)
`temp_1`	`temp-1`	Illegal character (-)
`AREA`	`AREA 1`	Illegal character (blank space)

Tips:
Assign an appropriate name for naming program element, by selecting word which depicts the function of the element.

C Language Introduction

Declaration

A statement to the compiler about the existence of a variable, constant, or function along with its data type, without having to provide an initial value.

example:

```
int umur;    // deklarasi variabel umur bertipe int
float luas; // deklarasi variabel luas bertipe float
```

C Language Introduction

Definition

A statement that not only declares but also allocates memory and usually provides an initial value.

example:

```
int umur = 20;
// definisi: variabel umur dibuat dan langsung diisi 20
```

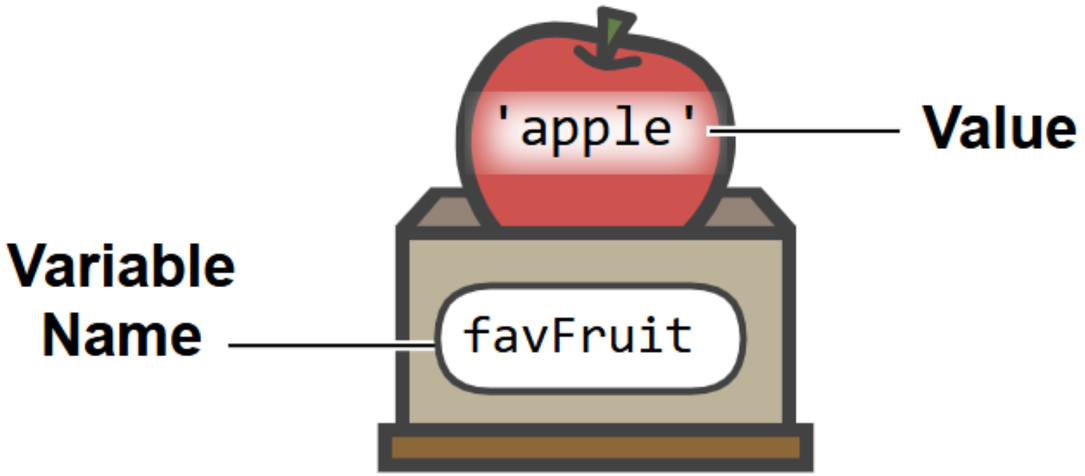
C Language Introduction

Variable

A variable is an identifier that is used to represent a single data item; i.e. a numerical quantity or a character

example:

```
int umur = 20;
umur = 25;
```



C Language Introduction

Constant

A fixed value that cannot be changed while the program is running.

integer constant:

```
const int MAX = 100;
```

C Language Introduction

Array (Preview)

Used to store a collection of several elements or values of the same data type in one place.

example:

```
int nilai[3] = {80, 85, 90};
```

```
nilai[0] = 80, nilai[1] = 85, nilai[2] = 90
```

C Language Introduction

String

- A string is a collection of characters that form text or sentences.
- In C, a string is represented as an array of characters.

example:

```
char nama[] = “Ali”;
```

THANK YOU!

REFERENCES

1.Beginner's Step-by-Step Coding Course: Learn Computer Programming the Easy Way. (2020). United Kingdom: Dorling Kindersley Limited.

2.Cornejo, N. (2020, May 4). Algorithms, Coding, and Programming. What’s the difference? Medium. https://medium.com/@web_trees/algorithms-coding-and-programming-whats-the-difference-bb83c8141220